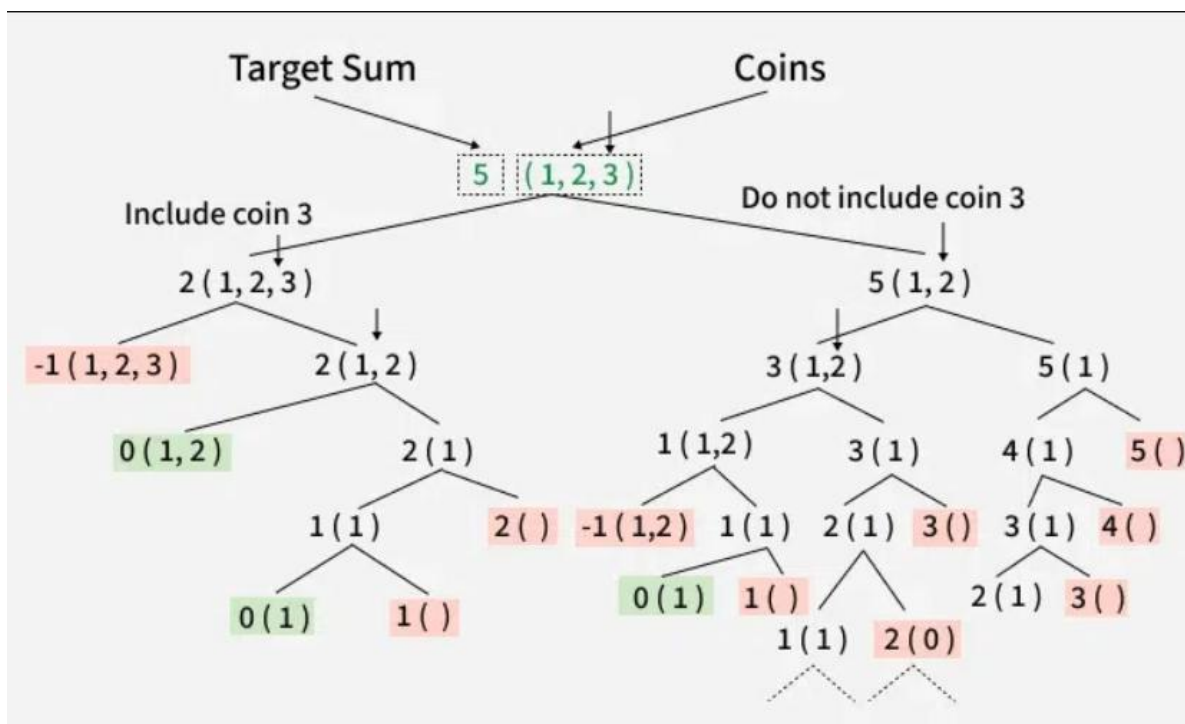


A. Recursion: Coin Change - Count Ways to Make Sum

Given an integer array of **coins[]** of **size n** representing different types of **denominations** and an integer **sum**, the task is to count all combinations of coins to make a given value **sum**.



Input:

sum = 10, coins[] = [2, 5, 3, 6]

Output:

5

Explanation:

There are five solutions:

[2, 2, 2, 2, 2], [2, 2, 3, 3], [2, 2, 6], [2, 3, 5] and [5, 5]



Exercise 1.

Given a set of coin denominations and a target sum, determine the total number of ways to make that sum using any number of coins (including zero).

The problem follows these base conditions:

1. If the target sum is 0, there is exactly one way to make the sum by choosing no coins.
 - Mathematically: $\text{count}(0, \text{coins}, n) = 1$
2. If the target sum becomes negative or no coins remain to consider ($n == 0$), there are no ways to make the sum.
 - Mathematically: $\text{count}(\text{sum}, \text{coins}, 0) = 0$

The goal is to compute the total number of combinations of coins that add up to the given target sum.

```
import time
coins = [1, 2, 3]
total = 5
dp = [0] * (total + 1)
dp[0] = 1 # one way to make sum 0 (no coins)
start_time = time.time()
i = 0
while i < len(coins):
    coin = coins[i]
    j = coin
    while j <= total:
        dp[j] += dp[j - coin]
        j += 1
    i += 1
end_time = time.time()

print(f"Coins: {coins}")
print(f"Target Sum: {total}")
print(f"Total number of ways: {dp[total]}")
print(f"Execution time: {end_time - start_time:.6f} seconds")
```



Exercise 2.

Given a set of coin denominations and a target sum, determine the total number of ways to make that sum using any number of coins (including zero).

The problem follows these base conditions:

1. If the target sum is 0, there is exactly one way to make the sum by choosing no coins.
 - Mathematically: $\text{count}(0, \text{coins}, n) = 1$
2. If the target sum becomes negative or no coins remain to be considered ($n == 0$), there are no ways to make the sum.
 - Mathematically: $\text{count}(\text{sum}, \text{coins}, 0) = 0$

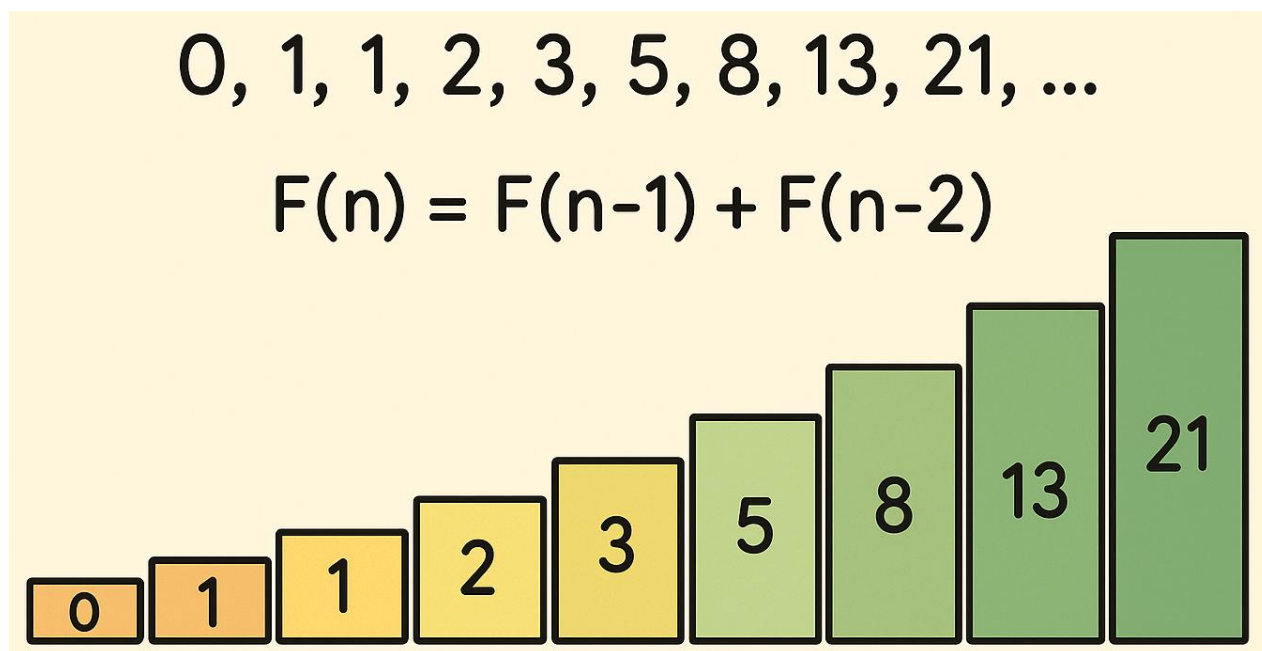
The goal is to compute the total number of combinations of coins that add up to the given target sum.

```
import time
def countRecur(coins, n, total):
    if total == 0:
        return 1
    if total < 0 or n == 0:
        return 0
    return countRecur(coins, n, total - coins[n - 1]) + countRecur(coins, n - 1, total)
def count(coins, total):
    return countRecur(coins, len(coins), total)
if __name__ == "__main__":
    coins = [1, 2, 3]
    total = 5
    start_time = time.time()
    result = count(coins, total)
    end_time = time.time()
    print(f"Result: {result}")
    print(f"Execution time: {end_time - start_time:.6f} seconds")
```



B. Recursion: Fibonacci sequence

To print the Fibonacci sequence in Python, we need to generate a series of numbers where each number is the sum of the two preceding ones, starting from 0 and 1. The Fibonacci sequence follows a specific pattern that begins with 0 and 1, and every subsequent number is the sum of the two previous numbers.



Input:

$$F(n) = F(n-1) + F(n-2)$$

Where:

$$F(0) = 0$$

$$F(1) = 1$$

$F(n)$ for $n > 1$ is the sum of the two preceding numbers.

Output:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...



Exercise 3.

Generate and print the first n terms of the Fibonacci sequence.

```
import time
n = 10
a = 0
b = 1
next = b
count = 1
start_time = time.time()
while count <= n:
    print(next, end=" ")
    count += 1
    a, b = b, next
    next = a + b
end_time = time.time()
print(f"\nExecution time: {end_time - start_time:.6f} seconds")
```

Exercise 4.

Generate and print the first n terms of the Fibonacci sequence. Where The first number is **0** and the second number is **1**.

Conditions:

If $n < 0$, the input is invalid.

If $n = 0$, the output should be 0.

If $n = 1$, the output should be 1.

For $n > 1$, compute the sequence iteratively until reaching the n th term.

The program should also measure and display the execution time taken to compute the result.

```
import time
def fibonacci(n):
    a = 0
    b = 1
    if n < 0:
        print("Incorrect input")
        return None
    elif n == 0:
        return 0
    elif n == 1:
        return b
    else:
        for _ in range(1, n):
            c = a + b
            a, b = b, c
        return b
start_time = time.time()
result = fibonacci(9)
end_time = time.time()
print(f"Result: {result}")
print(f"Execution time: {end_time - start_time:.6f} seconds")
```

Exercise 5.

Base case:

If n is 0 or 1, return n (since Fibonacci (0) = 0, Fibonacci (1) = 1).

Recursive case:

Each term is the sum of the two previous terms:

$$F(n) = F(n - 1) + F(n - 2)$$

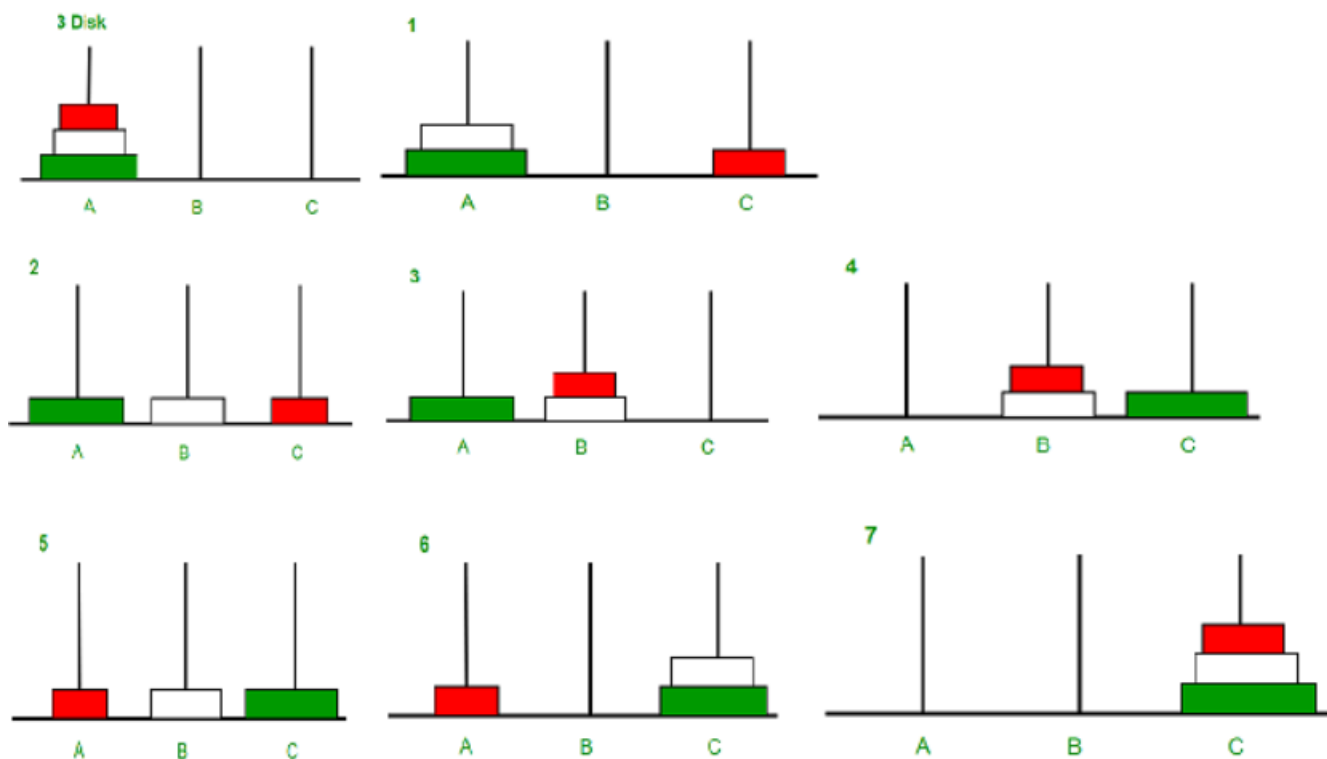
```
import time
def fibonacci(n):
    if n <= 1:
        return n
    else:
        return fibonacci(n - 1) + fibonacci(n - 2)
num = int(input("Enter the number of terms: "))
start_time = time.time()
for i in range(num):
    print(fibonacci(i), end=" ")

end_time = time.time()
print(f"\nExecution time: {end_time - start_time:.6f} seconds")
```

C. Recursion: Tower of Hanoi Algorithm

The Tower of Hanoi is a mathematical puzzle where we have three rods (**A**, **B**, and **C**) and **N** disks. Initially, all the disks are stacked in order of decreasing diameter, i.e., the smallest disk is placed on top, and they are on rod **A**. The objective of the puzzle is to move the entire stack to another rod (here considered **C**), obeying the following simple rules:

- Only one disk can be moved at a time.
- Each move consists of taking the upper disk from one of the stacks and placing it on top of another stack i.e. a disk can only be moved if it is the uppermost disk on a stack.
- No disk may be placed on top of a smaller disk.



Exercise 6.

Write a recursive function `towerOfHanoi(N, from_rod, to_rod, aux_rod)` to solve the Tower of Hanoi problem.

The Tower of Hanoi puzzle consists of **three** rods and N disks of different sizes that can slide onto any rod. The puzzle starts with the disks neatly stacked in ascending order of size on one rod, the smallest at the top.

The goal is to move the entire stack to another rod following these rules:

- Only one disk can be moved at a time.
- Each move consists of taking the top disk from one stack and placing it on top of another stack.
- No larger disk may be placed on top of a smaller disk.

Steps:

- Move the top $N-1$ disks from the source rod to the auxiliary rod.
- Move the remaining (largest) disk from the source rod to the destination rod.
- Move the $N-1$ disks from the auxiliary rod to the destination rod.
- The program should also measure and print the execution time for solving the puzzle.

```
import time
def TowerOfHanoi(n, from_rod, to_rod, aux_rod):
    if n == 0:
        return
    TowerOfHanoi(n - 1, from_rod, aux_rod, to_rod)
    print(f"Move disk {n} from rod {from_rod} to rod {to_rod}")
    TowerOfHanoi(n - 1, aux_rod, to_rod, from_rod)

N = 3
start_time = time.time()
TowerOfHanoi(N, 'A', 'C', 'B')
end_time = time.time()

print(f"\nExecution time: {end_time - start_time:.6f} seconds")
```

Exercise 7.

The Tower of Hanoi puzzle consists of **Four** rods and N disks of different sizes that can slide onto any rod. The puzzle starts with the disks neatly stacked in ascending order of size on one rod, the smallest at the top.

The goal is to move the entire stack to another rod following these rules:

- Only one disk can be moved at a time.
- Each move consists of taking the top disk from one stack and placing it on top of another stack.
- No larger disk may be placed on top of a smaller disk.

Steps:

- Move the top N-1 disks from the source rod to the auxiliary rod.
- Move the remaining (largest) disk from the source rod to the destination rod.
- Move the N-1 disks from the auxiliary rod to the destination rod.
- The program should also measure and print the execution time for solving the puzzle.

```
import time

def TowerOfHanoi(n, from_rod, to_rod, aux_rod):
    if n == 0:
        return
    TowerOfHanoi(n - 1, from_rod, aux_rod, to_rod)
    print(f"Move disk {n} from rod {from_rod} to rod {to_rod}")
    TowerOfHanoi(n - 1, aux_rod, to_rod, from_rod)

N = 4
start_time = time.time()
TowerOfHanoi(N, 'A', 'C', 'B')
end_time = time.time()

print(f"\nExecution time: {end_time - start_time:.6f} seconds")
```

Exercise 8.

The Tower of Hanoi puzzle consists of **Four** rods and N disks of different sizes that can slide onto any rod. The puzzle starts with the disks neatly stacked in ascending order of size on one rod, the smallest at the top.

```
import time
N = 3
rods = {'A': list(range(N, 0, -1)), 'B': [], 'C': []}
def move_disk(from_rod, to_rod):
    disk = rods[from_rod].pop()
    rods[to_rod].append(disk)
    print(f"Move disk {disk} from rod {from_rod} to rod {to_rod}")
start_time = time.time()
# If number of disks is even, swap destination and auxiliary rods
if N % 2 == 0:
    dest, aux = 'B', 'C'
else:
    dest, aux = 'C', 'B'

total_moves = 2 ** N - 1
moves = 1
while moves <= total_moves:
    if moves % 3 == 1:
        # Move between source and destination
        if not rods['A'] or (rods[dest] and rods['A'][-1] > rods[dest][-1]):
            move_disk(dest, 'A')
        else:
            move_disk('A', dest)
    elif moves % 3 == 2:
        # Move between source and auxiliary
        if not rods['A'] or (rods[aux] and rods['A'][-1] > rods[aux][-1]):
            move_disk(aux, 'A')
        else:
            move_disk('A', aux)
    elif moves % 3 == 0:
        # Move between auxiliary and destination
        if not rods[aux] or (rods[dest] and rods[aux][-1] > rods[dest][-1]):
            move_disk(dest, aux)
        else:
            move_disk(aux, dest)
    moves += 1
end_time = time.time()
print(f"\nTotal moves: {total_moves}")
print(f"Execution time: {end_time - start_time:.6f} seconds")
```